

3/3/2026

unit-2

Problem Solving Techniques by Searching

problem
formulation
(Designing)

goal (solution)
formulation

Informed (known) uninformed (unknown)

Problem Solving can be done in Search Techniques

> known

> unknown

* Problem Solving Agent does this

am
(or)
p-b

Problem Solving Agent :- It is one type of a goal based Agent which has 2 things pre determine goals
goal formulation and Problem formulation

↓

property

*
property

(situation)
current state and agent Performance

Problem formulation is a process which decides which action takes place (or) should be taken for a state to reach goal.

* →

(Algorithm:-

Problem Solving Agent (Percept) return actions

Persistent →)

Properties:-

1. Search:- It is an Algorithm which takes Input and returns Solution or Output
2. Solution - It is a Result (or) Output of Search
3. Execution - The Action which gives the Solution is called execution

→ Algorithm:-

Problem Solving Agent (Percept) return actions

Persistent → Sequence - (Action) Sequence of action - Initially Null

State - current state of world

Goal - (Destination) Desired goal

Problem - Formulation of Problem

State - update - State (State, Percept)

If Sequence is Empty

Goal - Formulation (State)

Problem - Formulation (State, goal)

Sequence Search - (Problem)

eg Linear search - Normal seq

Binary seq - Random seq

If Sequence = failure then

return null action (search unsuccessful)

Action - First (Sequence) (search successful)

Sequence - Rest (Sequence)

Return Action

→ well defined Problem:-

It is a structured Problem which may have solution is called well defined Problem.

Properties:-

1) Initial State:- The Starting State.

2) Action:- Description of possible actions available to Agent which may lead to solution. (path shortest)
(xle Actions - only the 1 will be executed)

3) Applicable:- which action is applicable, the action which is executable

4) Transition Model:- combination of Initial State and Action, which should gives the result

5) Successor:- It is a given action from one State to another State. Transition is taking place [shifting]
① Son is ⑤ Father
② is Successor

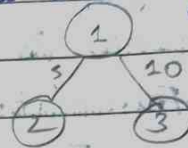
6) State Space - Initial State, Action, Transition Model

7) path - Network which constructs Directed (or)

①
②

undirected graph which is used for Problem Solution

- 8) path cost :- A Numeric value which is assigned to each path in a graph.
- 9) goal cost :- A cost which ensures a Path from S to S_1 through Action (S, a, S_1)
(Moving/Transition) Old State Action New State
- 10) Goal Test :- A State which reaches the Goal State Destination is goal



Shortest path $\rightarrow 1-2$

→ Example problems :- Is a well defined problem
 It is used for finding goals.

2 Types :-

- 1) Toy problem
- 2) Real world problem

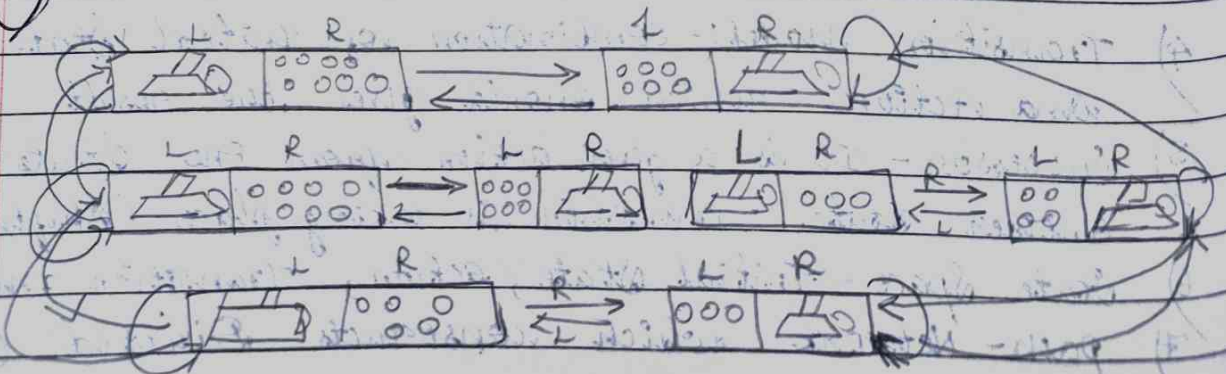
1) Toy problem :- It is one type of example problem which have 5 components and solution.

- | | | |
|-------------------|----------------------|--------------|
| i) State | iii) Action | v) Goal Test |
| ii) Initial State | iv) Transition Model | |

→ Toy problems :-

- 1) vacuum cleaner
- 2) Tiles
- 3) Queens problem

1) vacuum cleaner :-



i) State - n input, 2^n output $n=4$

ii) Initial State - Left

iii) Action - Left to right

right to left

Top Left to bottom left

Top Right to bottom right

Suck

iv) Transition model:- Initial State + Action
Left + Action(s)

v) Goal test:- All square should be clean

② ~~7~~ Tiles Problem:- there are tiles & inside that there are 9 tiles.
problem:-

7	3	1
4		6
5	2	8

It should be arranged in order way where 1st tile will be empty (or) last tile will be empty.

$n+1$ tiles are there
 n tiles is fill

①

②

Solution:

	1	2
3	4	5
6	7	8

1	2	3
4	5	6
7	8	

i) State:- $n+1$ tiles Input, n tiles Output

ii) Initial State:- unsorted tiles

iii) Action:- Left - right

right - left

Top - bottom

bottom - top

iv) Transition model:- unsorted to 4 Action

v) Goal :- n tiles are sorted in $n+1$ square

Initial "4" Input - 4 tiles

Initial square 3×3 with Initial

Input - 7 tiles

3) 8 Queens Problem:-

- * It is type of toy problem where there is condition.
- * No 2 Queens should be in same Row and No 2 Queens should be in same column.
- * No 2 Queens should be in same Diagonal and all Queens should be placed safely in the matrix.

4 Queens

			Q ₁		
	Q ₂				
				Q ₃	
		Q ₄			

Q₁ = {1, 3}
 Q₂ = {2, 1}
 Q₃ = {3, 4}
 Q₄ = {4, 2}

8 Queens

			Q ₁				
	Q ₂						
				Q ₃			
		Q ₄					

$$Q_1 = \{1, 3\}$$

$$Q_2 = \{2, 1\}$$

$$Q_3 = \{3, 4\}$$

$$Q_4 = \{ \quad \}$$

$$Q_5 = \{ \quad \}$$

$$Q_6 = \{ \quad \}$$

$$Q_7 = \{ \quad \}$$

$$Q_8 = \{ \quad \}$$

1) State - n Input 2ⁿ Output

2) Initial State:- 8x8 empty square matrices

3) Action:- Left - Right

- right-left
- Top-bottom
- bottom-top

- 4) Transition Model:- 8×8 empty Matrix + 4 Action
- 5) Goal:- All Queen should be placed safely in Matrix.

2) → Real world Problem:-

It is one type of example problem used for real world Problem Solution like Shortest Route by using Dijkstra's Algorithm, Networking Problem (LAN, WAN, MAN), Travelling Sales Man Problem, Airport-flight-current time-Location

→ 5 components:-

- 1) State
- 2) Initial State
- 3) Action
- 4) Transition Model
- 5) Goal

For Eg:- In Airport Management System

1. State:- current time, location
2. Initial State:- Airport starting location where flight is going take off
3. Action:- Reservation - Seat booking, from where booking require
- class & based on current location.
4. Transition Model:- Initial State + Action
5. Goal:- Specified Destination.

→ Solution by Search/Searching for Solution:-

Search is a technique which applies some Algorithm to get the solution for a problem.

→ Parameters for Search / I categories:-

1. Search tree

2. Node

3. Expanding

4. Generating

5. Parent

6. Child

7. Leaf node

8. frontier node

9. Open list

1) It is one type of tree which has branches and a starting node called root Node and uses some Action (Traversal)

2) It is Initial State (Root)

3) Expansion is process where tree get expand to subtree which consist parent node & child Node.

4) Generating is a process which is used for creating new node for state

5) Parent is a node, which is Predecessor of child node, root is always parent node.

6) Child is Successor of parent Node

7) Leaf node is a node doesnot have child node.

8) frontier node is a node where a leaf node is ready for expansion.

9) The path b/w Source & destination used for Searching is called Open list.

- 2nd categories:-
1. Search Strategy
 2. Repetative State
 3. Loop path
 4. Redundant path

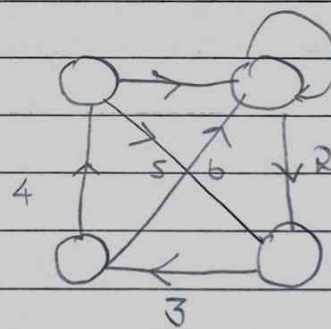
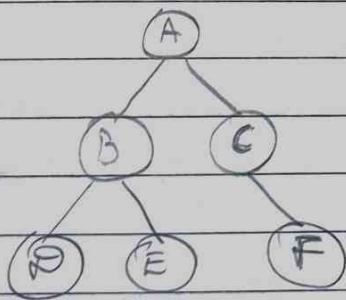
1. The Search Strategy is a Technique which is reduced from One problem to another problem.

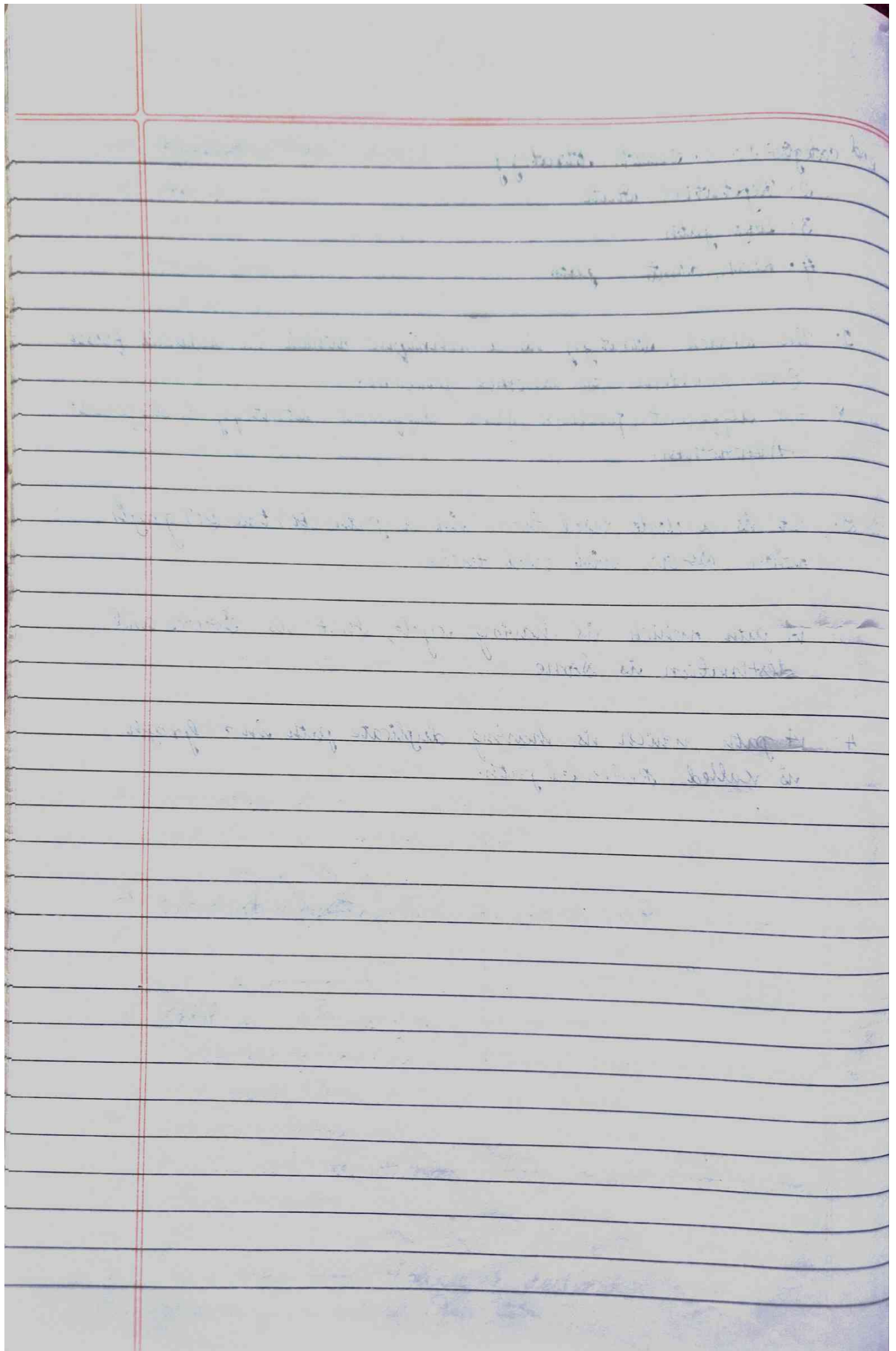
* for different problem have different Strategy & different Algorithm.

2. It is a Node used twice in a path (or) tree (or) graph. where source will used twice.

3. A path which is having cycle, that is source and destination is same

4. A path which is having duplicate path in a graph is called Redundant path.

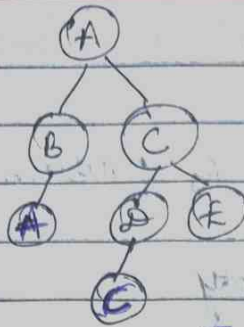




9/2/26

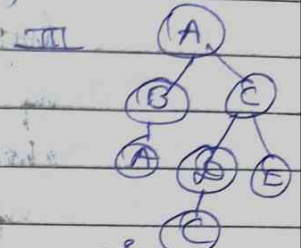
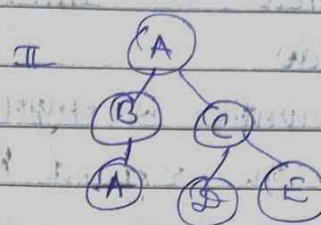
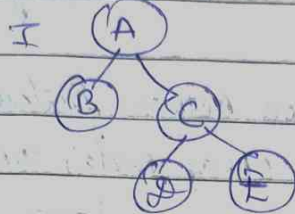
SPARK

Date _____ Page _____



Leaf Node (No child) → Got expanded
get expanded Frontier Node

A is Leaf Node → A is Goal



A is source & A

C is source & C

is only destination

is only Destr

2 Goal Nodes → A, C

3 Leaf Nodes → A, C, E

Algorithm:

TREE - SEARCH - ALGORITHM (Problem) return solutions

Initialize

(Root)

Frontier Node from the Initial State of problem

Loop do

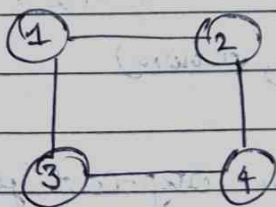
* if Frontier is empty then Return failure

Leaf Node

choose the expand Node (Leaf Node) and ^{Remove} add to frontier

* if the chosen Node is a goal state then add it

* to the Result (or) solution and Return solution



here 1 is source then

3 will become frontier

1 will be goal

Empty

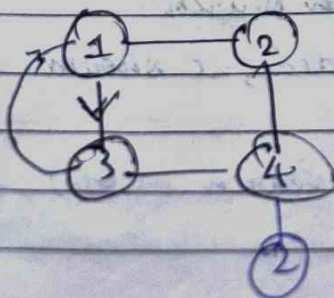
I

II

$f = \{ \}$
 $FS = \{ \}$

$f = \{ 3 \}$
 $FS = \{ 1 \}$
Goal

$f = \{ 3, 4 \}$
 $FS = \{ 1, 2 \}$



State, Initial, Action, Goal

$f = \{ \}$
 $ES = \{ \}$

both are empty

GRAPH-SEARCH-ALGORITHM (problem)

Initialize

Frontier node from the Initial State of problem

Initialize

↔ explored Set Initialize to be empty
loop do

if frontier is empty then Return failure

choose the expand Node (leaf node) and Remove to frontier

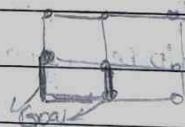
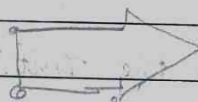
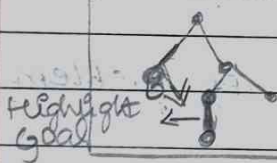
if the chosen node is a goal state then add it to result (or) solution and Return solution.

↔ Add the chosen Node to explore Set.

↔ If it is ^{not in} frontier (or) explored Set.

→ Solution State Space:-

Initial State, Action, Transition Model



* Goal state should be highlighted

Skeleton → matching with Tree & Graph

→ child-function (Problem, Parent, Action) returns a solution as a Node

Initial State ← State = Problem, Result (parent, State, Action)

Parent = Parent, action = action expansion (Goal Node)

Path-cost = Parent.Path-cost + Problem - Step-cost (parent, State, etc)

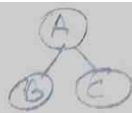
Informed

→ uniform Search Strategies / blind search:

A Node is going to be expanded, it should reach to goal node.

* It is also called Informed Search (or) heuristic Search.

for each - Group of Elements
for - only 1



1st Root is expanded
then 1st Successor (B) is expanded

SPARK

Date _____ Page _____

1) Breadth First Search:- It is one type of uniform search strategies where a node is expanded first from the root as Successor.

* This is based on FIFO-queue.

* The expanded node is called Shallow Node.

→ Breadth-first-search (Problem) returns a solution on Failure
Node → a node with State = Problem. Initial State. pathcost = 0

If Problem. Goal test (node, State) that returns solution (node)
Frontier ← FIFO queue C with only node as element
Explored ← Empty Set

* Loop do

if empty (Frontier) then returns failure

node → pop (Frontier)

Add node. State to Explore Set

for each ^{expansion} action in Problem. actions (node, State) do

child ← child Node. C ^{expansion} (problem, Node, Action) ^{exp → Goal → Set → last Node}

if child State is not in Frontier (or) Explored Set
then

If Problem. Goal test (child, State) that returns solution (child)

* Frontier ← Insert (child, Frontier)

Time complexity = $b + b^2 + b^3 + \dots + b^d = O(b^d)$

Ex:-

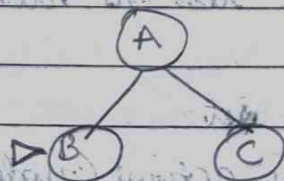
1)

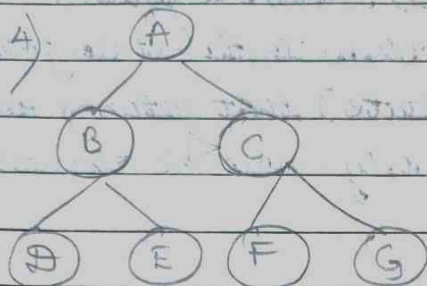
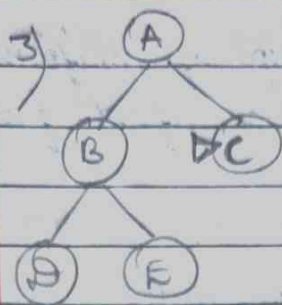


F = {B, C}

ES = {B, C, G}

2)





2) → uniform-cost search:-

It is 1 type of uniform-search strategy which should have uniform cost (or) Least cost.

* Also called as Least-cost Algorithm.

uniform-cost-search (problem) returns a solution or failure
node → a node with state = Problem, Initial state, path cost = 0

If problem.goal test (node.state) that returns solution (node)

Frontier ← FIFO queue with only node as element

Explored ← empty set

* Loop do

if empty? (Frontier) then return failure
node → pop (Frontier)

add node.state to explore set

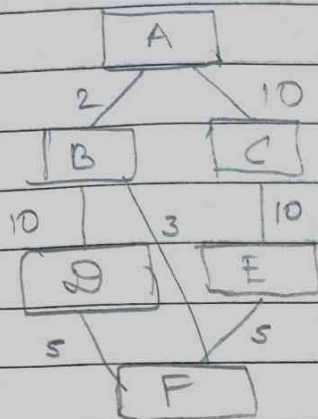
for each action in Problem.actions (node.state) do

child ← child Node (problem, Node, Action)

if child state is not in Frontier (or) explored set then
If problem.goal test (child.state) that returns solution (child)

- * $\text{Frontier} \leftarrow \text{Insert}(\text{child}, \text{Frontier})$
 else if child.State is in Frontier with high path cost
 then replace Frontier with child Node .

→ example of uniform-cost search



$O(b^{d+1})$

shortest path ABF

B is Frontier

while F is Frontier

because B gets replaced with F

3) Depth first - Search:- It is one type of uniform search which is based on depth of Node, once the depth is reached backtracking is required.

- * It is based on LIFO Algorithm

- * Frontier is there

Depth-First-Search (problem) returns Solution (or) failure

$\text{Node} \leftarrow$ A node with $\text{State} = \text{Problem}$. Initial State, path cost = 0

If $\text{problem.Goal test}(\text{node.State})$ that returns Solution (node)

$\text{Frontier} \leftarrow$ LIFO ^{stack} ~~queue~~ with node as only element

$\text{Explored Set} \leftarrow$ Initially empty set

- * Loop do

if empty? (Frontier) then returns failure

node $\rightarrow$ pop (Frontier)

add node.State to Explored Set

for each action in $\text{Problem.Actions}(\text{node.State})$ do

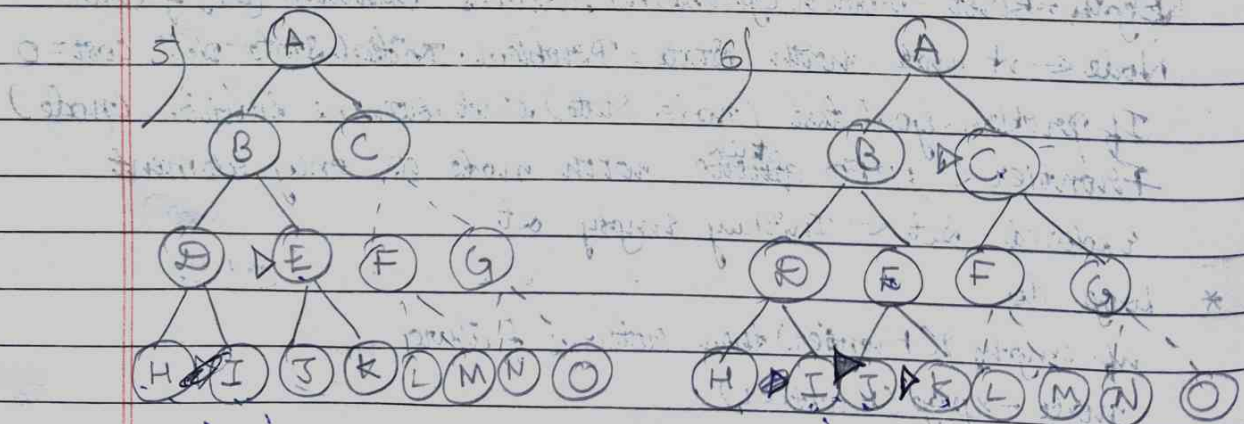
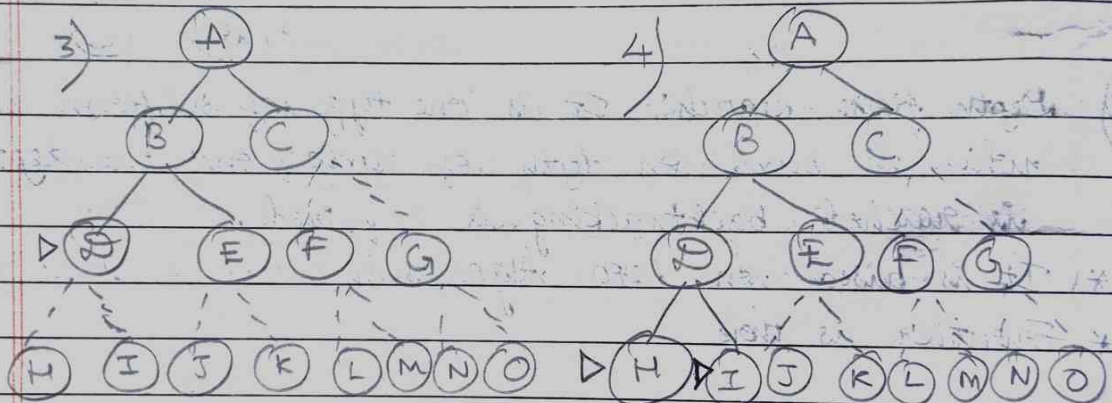
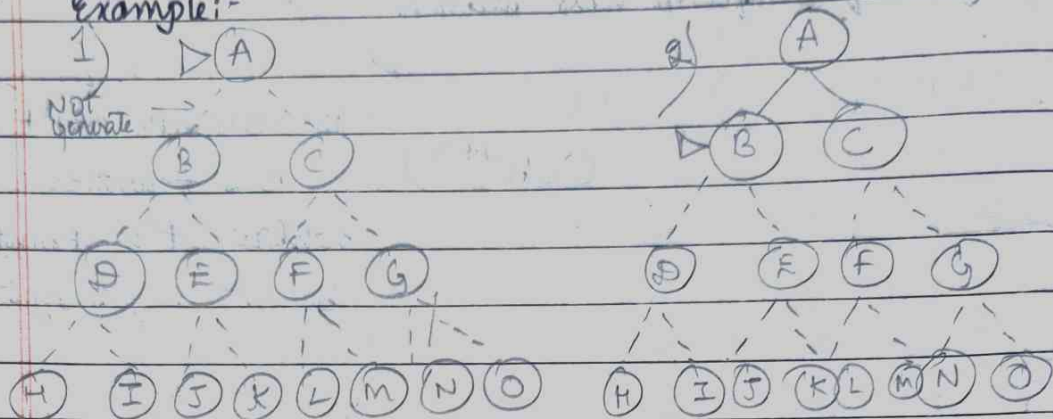
child $\leftarrow$ child Node (Problem, Node, Action)

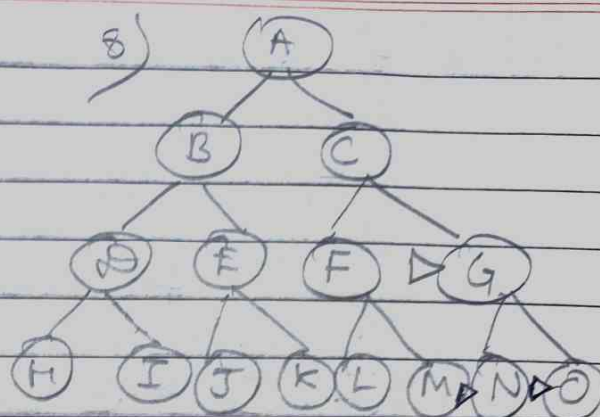
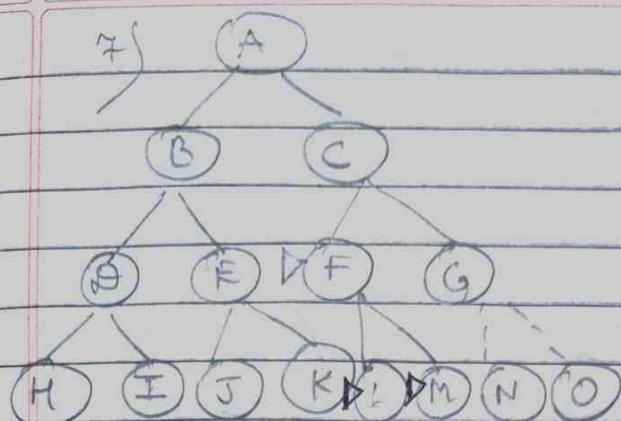
if child State is not in Frontier (or) Explored Set then

- * If `problem.goaltest(child.state)` that returns `Solution (child)`
- * `Frontier ← Insert (child, Frontier)`

→ Time complexity → $O(b^d)$

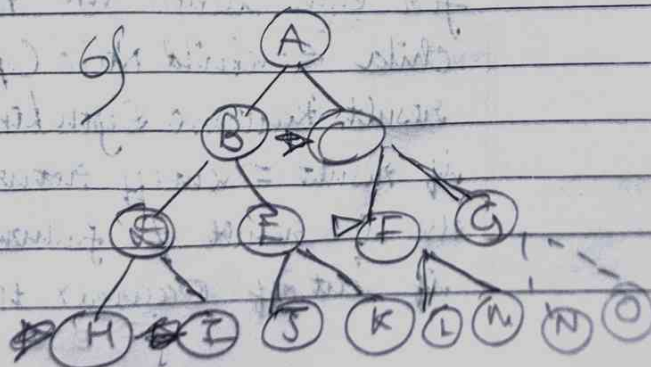
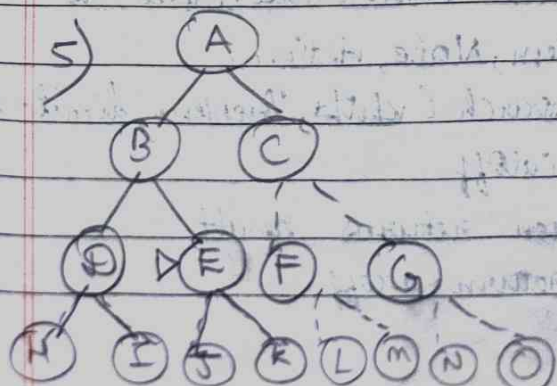
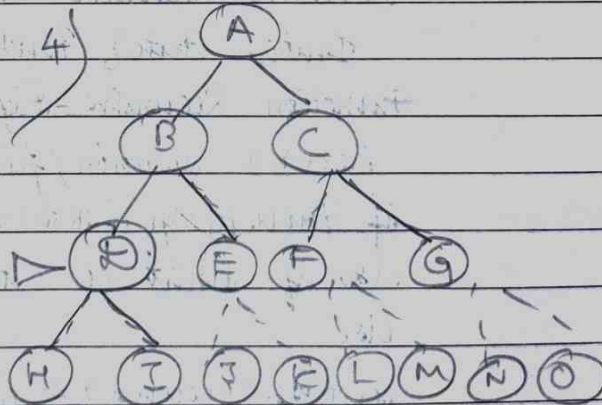
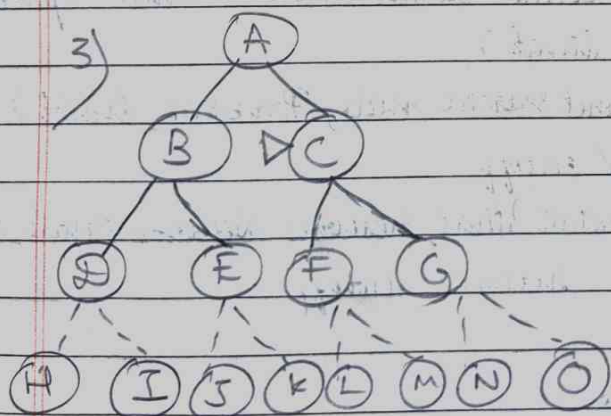
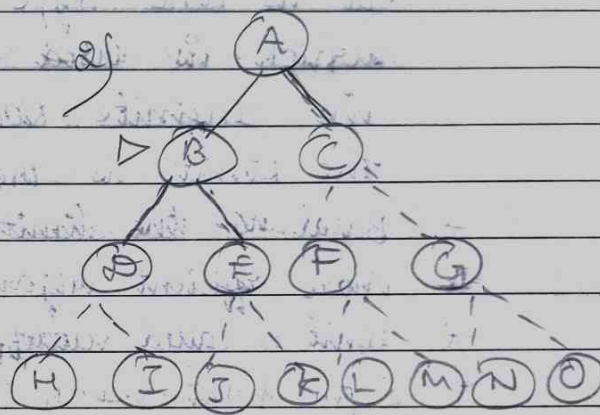
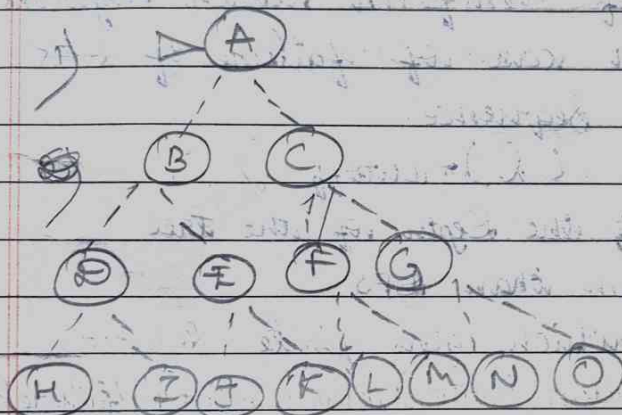
→ Example:-

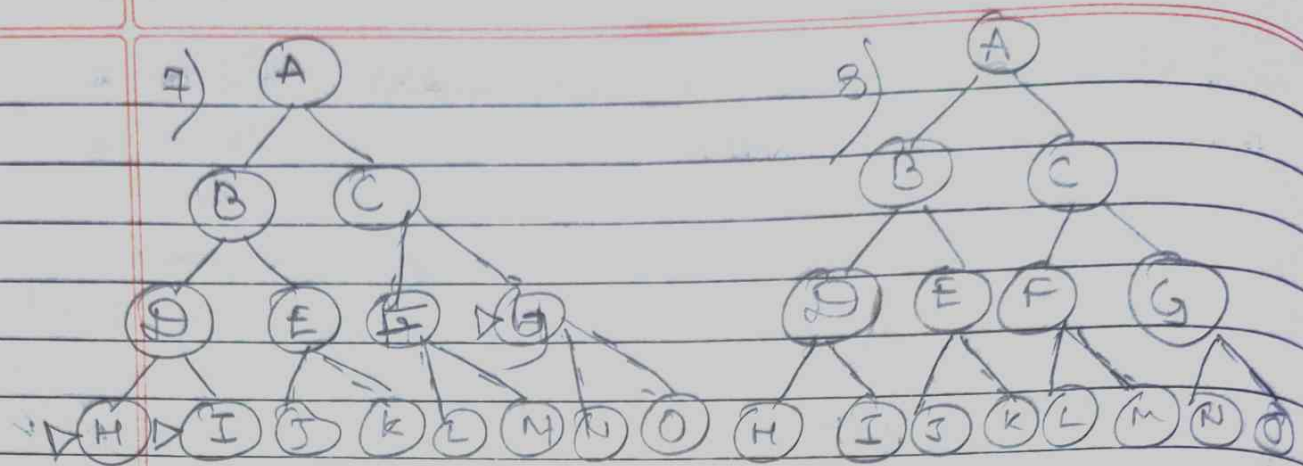




~~AB @ f i l e s~~

Order $\rightarrow A B \textcircled{+} H I \textcircled{+} J K C F L M G N \textcircled{+}$





→ Depth Limit Search: (where DFS fails)

It is one type of uniform Search Algorithm which is used in case of failure of DFS in Infinite Node Sequence.

The Limit is there (L) - cutoff

- * Based on the limit of the Depth of the tree
- * More efficient Algorithm than DFS
- * Limit is called cutoff which gives finite solution.
- * Depth-limit-Search (Problem, limit) returns solution/failure/cutoff
- * returns Recursive Depth Limit Search (make a node (problem, Initial State), Problem, limit)

Function Recursive-Depth Limit Search (node, Problem, limit)

returns solution/failure/cutoff

if Problem.goal test (node.State) that returns solution (node)

else if limit = 0 then return cutoff

else

cutoff-occured? ← failure

for each action in Problem.action (node.State) do

child ← child Node (problem, Node, Action)

result Recursive Depth Limit Search (child, Problem, limit-1)

if result = cutoff returns cutoff

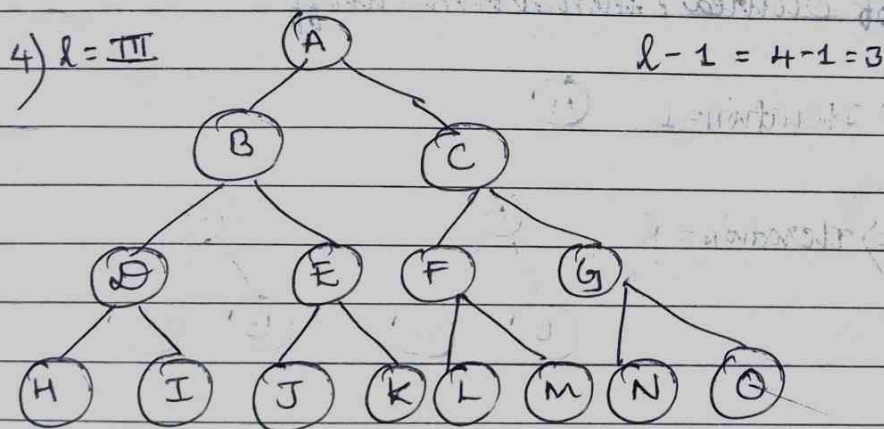
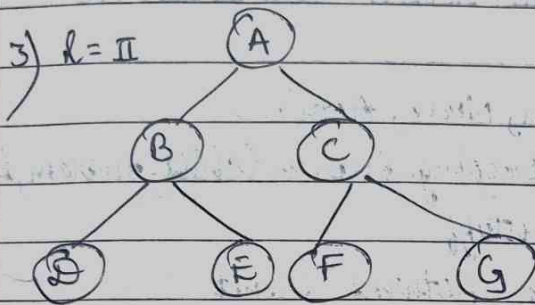
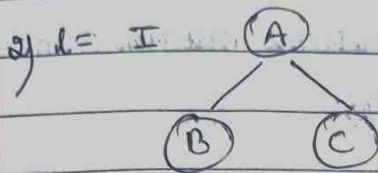
else if result ≠ failure then returns result

if cutoff Occured? then return cutoff

→ DLS

else
 returns failure (null)

eg:- 1) $l = 0$ (A)



* It should not exceed the cutoff while generating the tree.

Time complexity:- $O(b^l)$

→ Iterative Deepening- Search:-

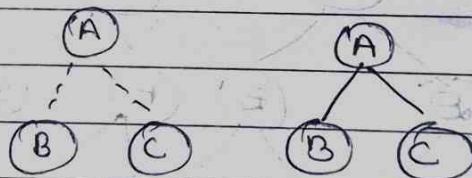
It is one type of uniform search strategy which is based on deeping of node in a tree.

- * It is a Iterative method i.e every iteration 1 Tree should be generated.
- * It is combination of DFS and DLS

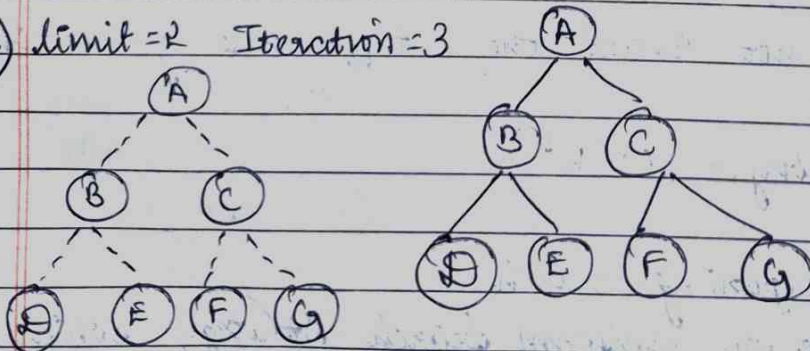
→ Iterative-Deepening-Search (problem, limit) returns Solution/failure/cutoff
 returns Recursive-Iterative-Deepening-Search (make a node (problem, Initial State), problem, limit)
 Function Recursive-Iterative-Deepening-Search (node, problem, limit)
 returns Solution/failure/cutoff
 if Problem.goal test (node.state) then returns Solution (node)
 else if limit = 0 then return cutoff
 else
 Cutoff-occurred? ← failure
 for each action in Problem.action (node.state) do
 Iteration
 child ← child Node (problem, Node, Action)
 result Recursive-Iterative-Deepening-Search (child, problem, limit-1)
 if result = cutoff returns cutoff
 else if result ≠ failure then returns result
 if cutoff occurred? then return cutoff

eg: 1) limit = 0 Iteration = 1 (A)

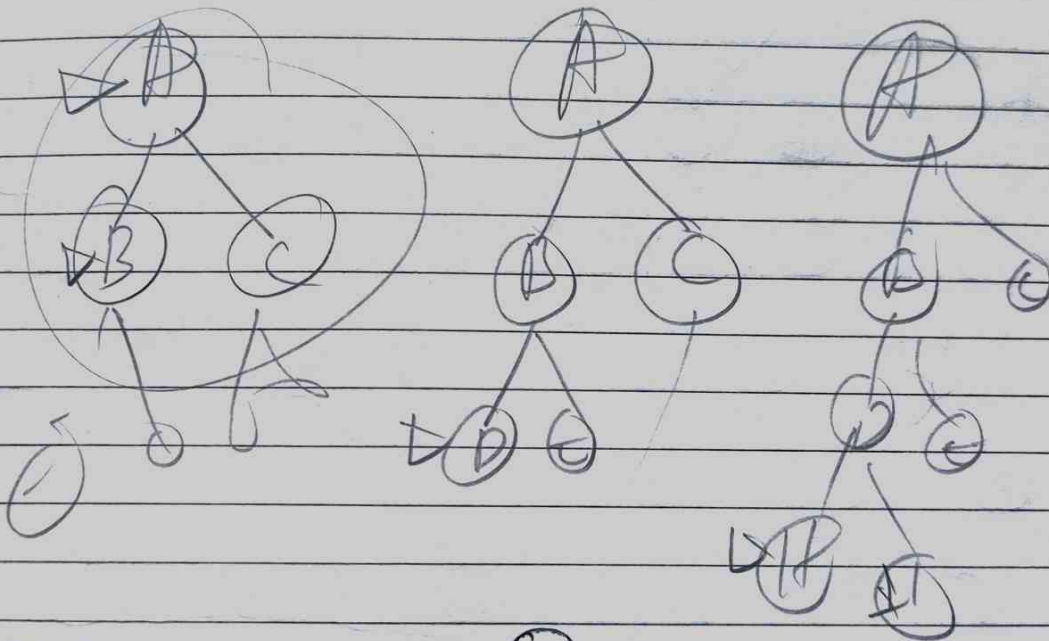
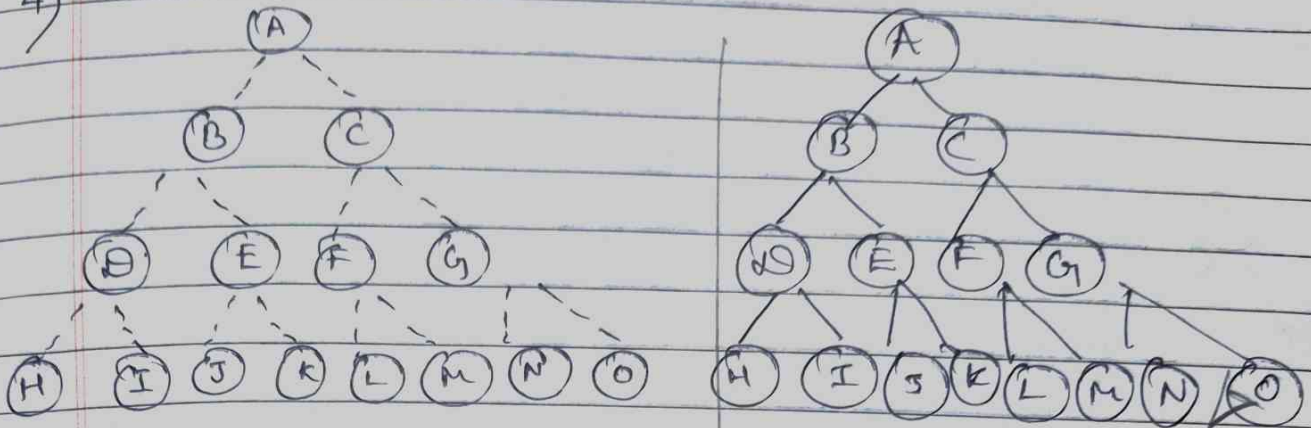
2) limit = 1 ⇒ Iteration = 2



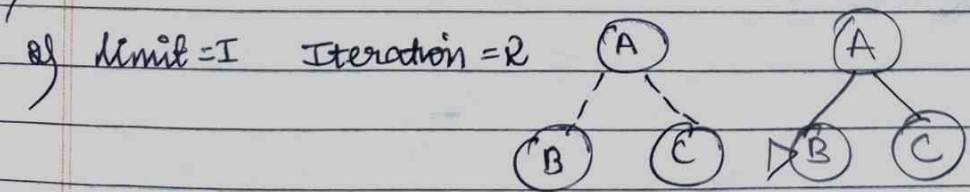
3) limit = 2 Iteration = 3



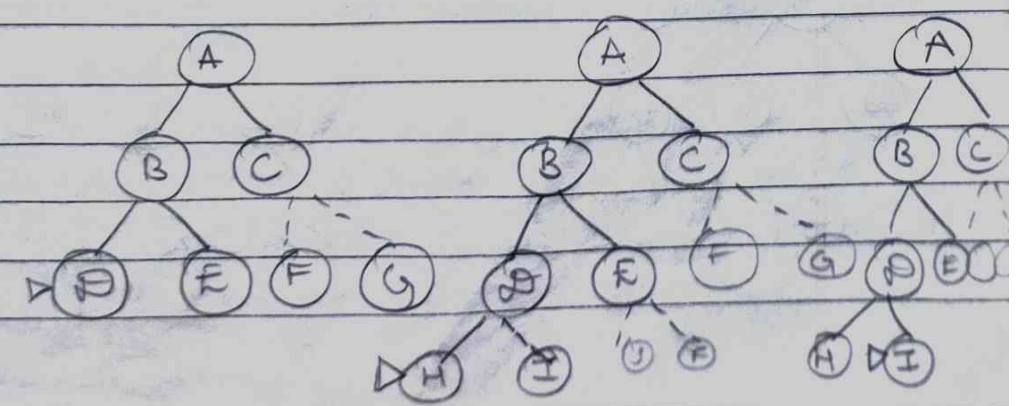
4) limit = 3 Iteration = 4

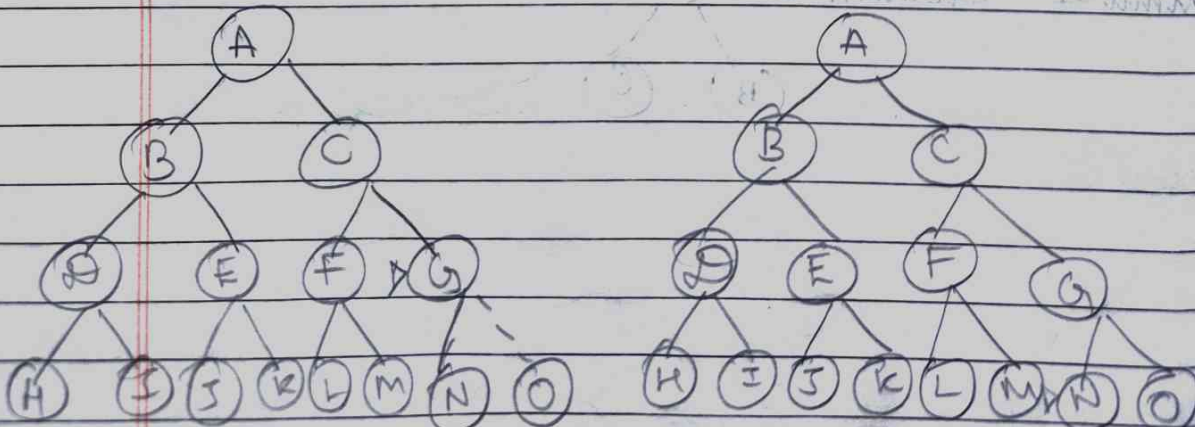
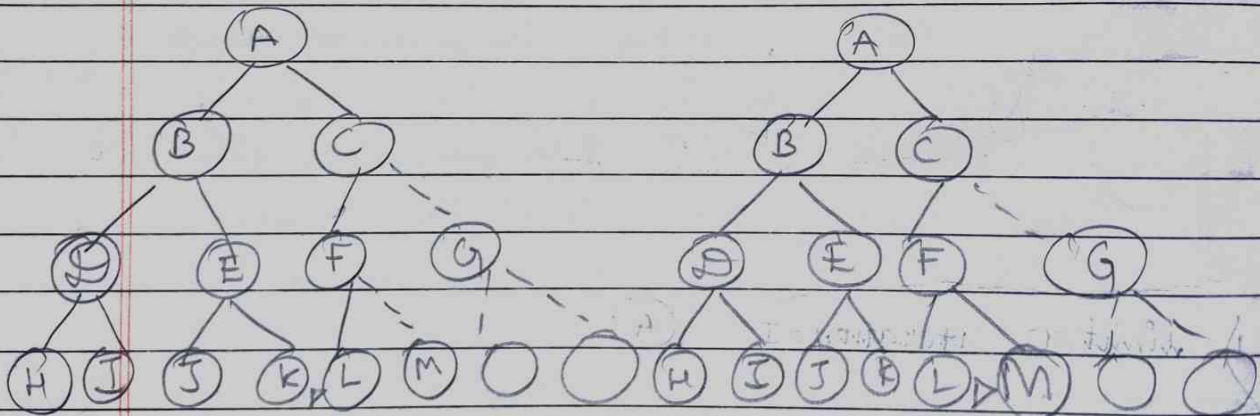
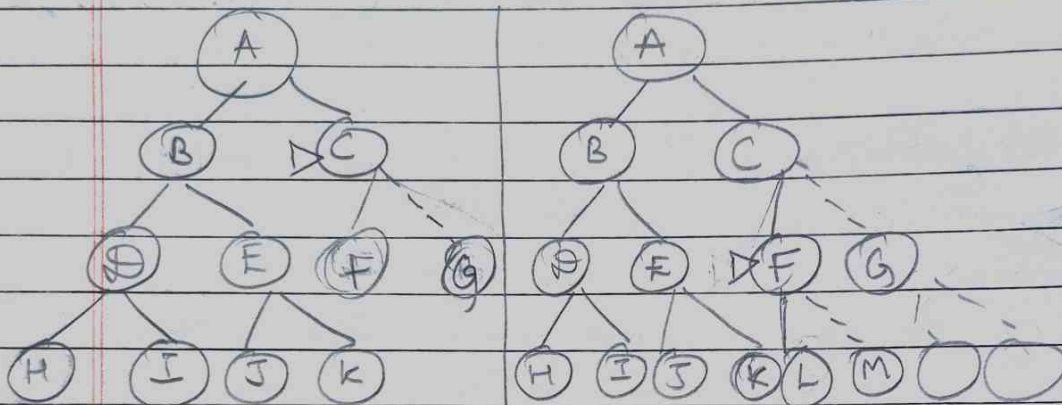
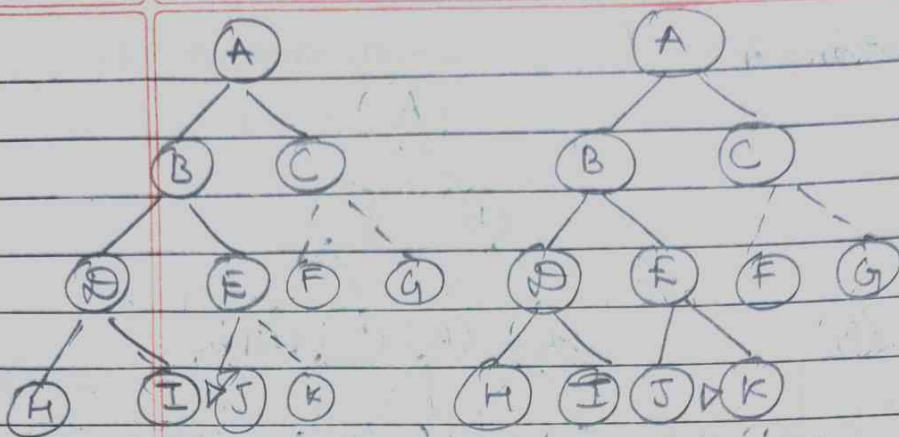


1) limit = 0 Iteration = I



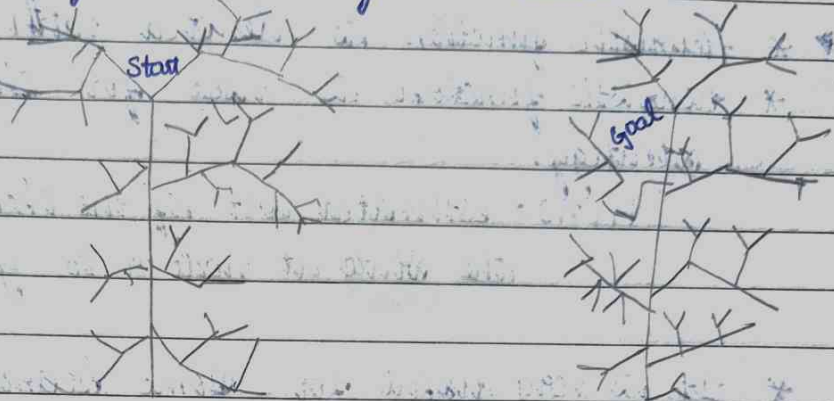
2) limit = I





→ Bidirectional Search :- uninformed

It is one type of search which starts from both end of a tree. (i.e. goal to) * should have 2 Queue
i.e. Start to goal and goal to start.



* At least 2 trees should be there | 'schematic view'
Start to goal → level → (Tree is called)
goal to start → depth ↓

* Time complexity $O(b^{d/2})$

Time complexity

<u>Informed search</u>			<u>uninformed</u>		
BFS	UCS	DFS	DLs	Deepening search	Bi-directionals
$O(b^d)$	$O(b^{d+1})$	$O(b^d)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$

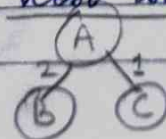
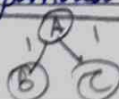
→ Heuristic Search :-

It is one type of Informed Search Strategy which is based on Tree and Graph Search Algorithm. (Based on Numeric value least cost should be found)

* Initially it uses BFS (FIFO) uses evaluative function, Heuristic Function

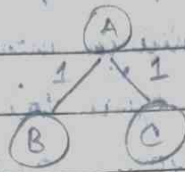
→ BFS :- where it is based on FIFO Algorithm.

→ evaluative function :- which is denoted with $f(n)$ which is used for evaluation of tree based on cost estimation and lowest cost is expanded first.



where c will be expanded

- * If 2 nodes are having same cost then it is based on FIFO where B comes first



- * Heuristic function is denoted with $h(n)$

AI will decide what to use to get optimal path

- * Heuristic function is used for choice of search strategy.

$h(n)$ = estimated cost of the cheapest path from the state at node n to goal state (n to g)

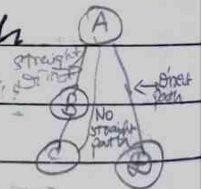
- * It is very useful for some search techniques

- 1) Greedy best-First Search:-

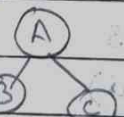
It is one type of heuristic search strategy which uses greedy technique where ' $h(n) = f(n)$ '.

- Properties:- based on least cost of $h(n)$ from source node it prioritizes the path

- ① * Straight line Distance:- The distance between source node to goal node must be a straight path

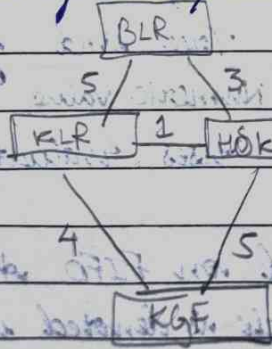


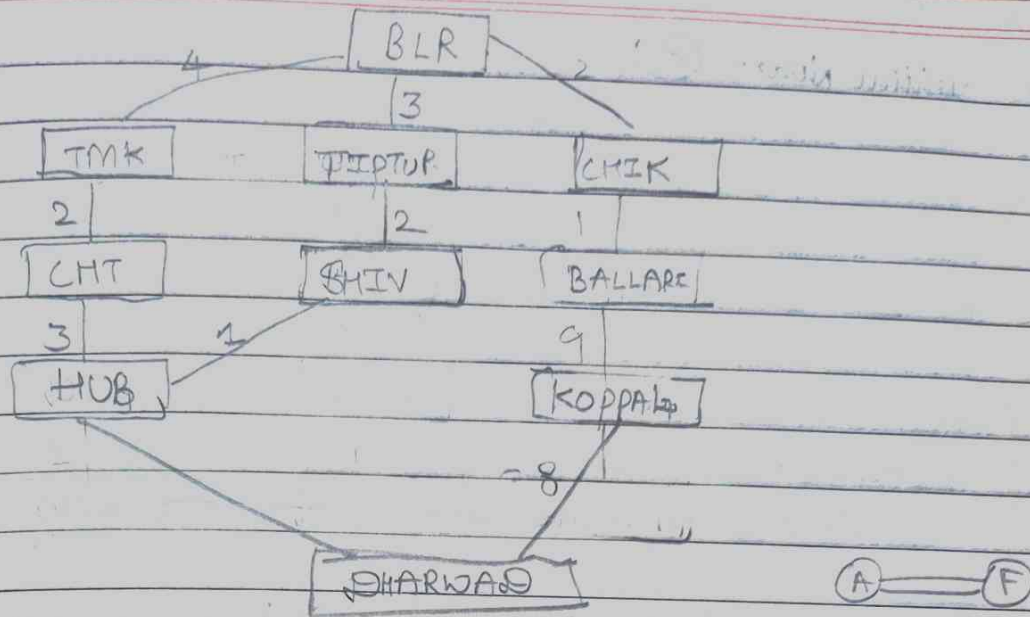
- * Greedy best first search strategy becomes Incomplete in finite state space



- ② BFS ③ evaluative ④ heuristic

- * The greedy best-first search the aim is to reach the goal using best/least cost.





→ Bidirectional Search (Source, goal) returns solution (or) failure

I create 2 Queues → Q_f Q_b

Q_{front} Q_{back}

II Input:- start in Queue front

III Input:- goal in Queue back

IV Mark the start node and goal Node visited

V while Queue front and Queue back is not empty

Expand from Queue front

if node found in backward is visited Node
return Path

expand from Queue back

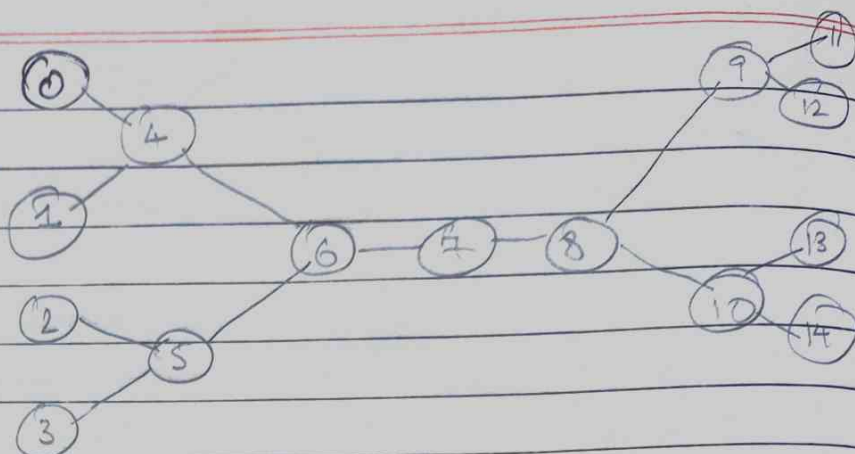
if node found in forward is visited Node
return Path

end while

returns Failure

eg-

Initial Node - 0



0-4-6-7-8-10-14

14-10-8-7-6-4-0

→ Greedy best first search (Source, goal) returns solution or failure

Open = Start

Destn

closed = { }

* while Open is not empty

1) n = node in Open with path h(n)

1 → 2 → 3

2) remove n from Open

3) if n == goal

4) return path

5) add n to closed

* for each success s (source) of n then

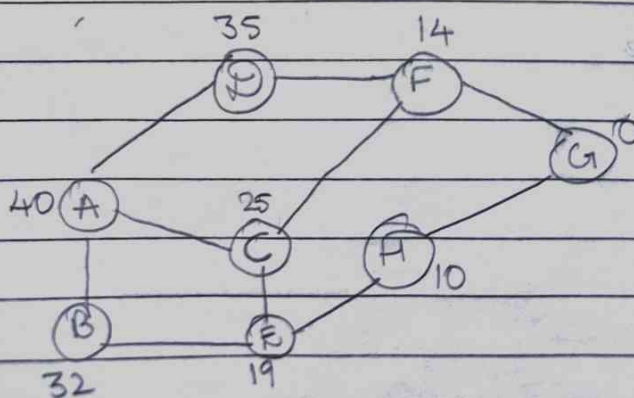
6) if s is not in open, not in closed

7) compute h(n)

8) add s to Open

if it not added

9) return failure



Destn → G = by '0'

I Source - A

A to D heuristic value $h(n) = 35$

A to C $h(n) = 25 \checkmark$

A to B $h(n) = 32$

Best path A to C = 25

II C to F $h(n) = 14$

C to E $h(n) = 19$

Best path C to F = 14

III F to G $h(n) = 0$
 $F \xrightarrow{0} G$

Total path Best path

A $\xrightarrow{25}$ C $\xrightarrow{14}$ F $\xrightarrow{0}$ G = (39)

→ Time complexity is $O(b^d)$

→ A* Algorithm:-

It is one type of informed search algorithm has $h(n)$ and $g(n)$ values and finds the least cost from source to goal.

$h(n)$ = heuristic value

* estimated cost from node n to goal.

$g(n)$ = Actual cost from start to (end) n.

$$f(n) = h(n) + g(n)$$

Algorithm:-

A* (start, goal)

1) Open = {start}

2) closed = {}

3) while Open is not empty

least cost → 4) n = node in Open with lowest $f(n)$

Reached last node

5) if $n == \text{goal}$ then

6) return path

7) remove n from Open

(stop) 8) Add n to closed

Computations for each Success (S) in n

10) $g(S) = g(n) + \text{cost}(n, S)$

11) $f(S) = g(S) + h(S)$

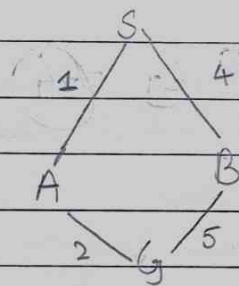
12) if S is not in Open and not in closed

13) add S to Open

14) else

15) return failure

eg:-



Node	$h(n)$
S	5
A	2
B	3
G	0

Step I:- Start (S)

	g	h	f
S	0	5	5

$$f = g + h$$

Step II:- Expand (S)

Node	g	h	f
A	1	2	3
B	4	3	7

Step III: $\delta^1 A^2 G = 3$
 $f(n) = 3.$

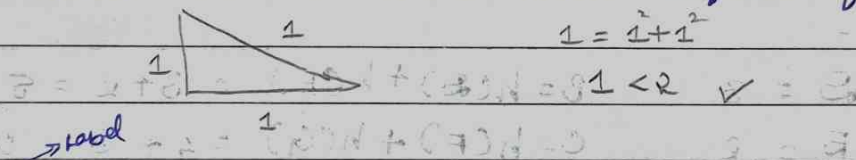
→ Time complexity is $O(b^d)$.

→ Properties

1) Admissability - Admissable heuristic
 It should be based on heuristic function $h(n)$
 $f(n) = g(n) + h(n)$

2) consistency - The graph should be consistent

3) Triangle Inequality - In Triangle, 1 side value should not be more than square of 2 side values



→ AO* Algorithm:- It is one type of Informed Search Algorithm which is based on AND-OR Graph used for finding Optimal Solution. * Requires Backtracking

* AND Node - Cost = Sum of all child costs (bottom)

Selection OR Node = Cost = Minimum of child cost (any 1)

(Bottom Approach)

Algorithm AO* (node) returns Solution (or) failure

if node = terminal node

mark solved

return cost

if node is OR node

for each child

compute cost

choose child with minimum cost

Root is OR always

if node is AND Node

for each child

compute cost

Cost = Sum of all childs

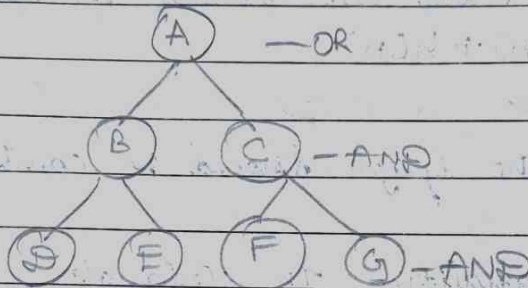
update node cost

if all required children solved

mark node solved

Repeat until root is solved

Eg:-



Heuristic values:- Solve Bottom to Top

→ Step 1:-

$$D = 3$$

$$B = h(D) + h(E) = 3 + 2 = 5$$

$$E = 2$$

$$C = h(F) + h(G) = 4 + 6 = 10$$

$$F = 4$$

$$G = 6$$

→ Step 2:- $A = \min(B, C)$

$$A = \min(5, 10)$$

$$A = 5$$

→ Step 3:-

$$A \rightarrow B(D \text{ and } E)$$

$$= 5 + 5 + 3 + 2$$

$$= 15$$

$$A \rightarrow C(F \text{ and } G)$$

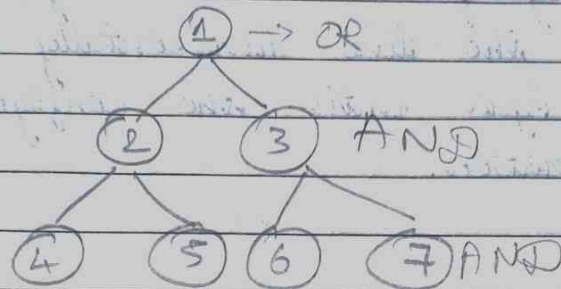
$$= 5 + 10 + 4 + 6$$

$$= 25$$

→ Step-4:-

The minimum cost path is $A \rightarrow B(D \text{ and } E)$
 $\therefore$ cost is 15

→ Time complexity is $O(b^d)$



S1:-

Heuristic values

4 = 5
5 = 6
6 = 9
7 = 4

$$2 = h(4) + h(5) = 5 + 6 = 11$$

$$3 = h(6) + h(7) = 9 + 4 = 13$$

S2:-

$$1 = \min(2, 3)$$

$$1 = \min(11, 13)$$

$$1 = 11$$

S3:-

$$1 \rightarrow 2(4 \text{ and } 5)$$

$$11 + 11 + 5 + 6$$

$$= 33$$

$$1 \rightarrow 3(6 \text{ and } 7)$$

$$11 + 13 + 9 + 4$$

$$= 37$$

S4:-

The minimum cost path is $1 \rightarrow 2(4 \text{ and } 5)$

cost is 33

numeric value which gives min cost

→ Heuristic Function:-

It is used for Solving Informed Search strategy where it is used for a ^{first} puzzle search problem.

The Outcome is current solution with random generated 8 puzzle instance is about 22 steps in the branching factor with 3 objects of the puzzle is to slide the tiles horizontally (or) vertically into empty space until the configuration matches goal configuration.

7	2	4			1	2
5		6			3	4
8	3	1			6	7

Initial state

Final state

$h1$ = The misplaced Tiles

$h2$ = ^{Sum of the distance} from Tiles to goal

The final state is achieved by applying steps to Initial State to reach the goal.

In 22 steps
we used
18 steps

The Manhattan distance from tiles 1 to 8

$$h2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$$

Generating Admissible heuristic from Sub Problems:
Pattern Database

→ Sub Problem:

Admissible heuristic can also be derived from the solution, cost of a subproblem of a given problem.

for eg:-

Remaining
4 No.
are
masked

★	2	4				1	2
★		★				3	4
★	3	1				★	★

Start State

Final State

- * The above diagram is a sub problem of the 8 puzzle Instance where 1 to 4 tiles are in correct position. The sub-problem involves not only the tiles in correct position also Optimal Solution for the sub-problem.

→ Pattern Database:-

(obj) The Idea behind pattern database is to store the exact solution cost for every possible sub problem instance. Every possible configuration of the 4 tiles and blank. The purpose of the sub problem is about the Heuristic Database "h_{DB}" for each complete state encountered during a search simply by looking up the corresponding sub-problem configuration in the Data Base. The Database itself constructed by search back from the goal and recording cost of each new pattern encountered. The Expense of this search is authorized in subsequent Problem Instances.

The choice of 1-2-3-4 is fairly arbitrary which connect database for 5-6-7-8 which can be combined by taking maximum value. A combine Heuristic of the Manhattan distance, the number of nodes generated when solving random 15 puzzles can be reduced by a factor of 1000.

when the Heuristic obtain from 1-2-3-4 Database and 5-6-7-8 is added to the Database without Overlay.
(Replacement)

→ DisJoint pattern Database:-

DisJoint pattern Database includes the sum of two costs which may be based on lower bound where number of nodes are reduced by a factor 10,000 compared with the use of Manhattan Distance.

→ Learning Heuristic from experience:-

A heuristic function $h(n)$ is supposed to estimate the cost of solution from beginning state at Node n . Another solution is learned from experience i.e. solving lot of 8 puzzles problems for instance each optimal solution to an 8 puzzle problem provides example from which $h(n)$ is learned.

(each example) *

Each example consist of a state from the solution path and the actual cost from the point based on predict solution cost from other states arises during the search. Techniques for doing this requires decision Trees and other Methods.

→ Features:- ^{which is based on something}

Inductive learning method is based on supplied feature of a state i.e. relevant to predict the state value rather than raw state description where for example if 100 randomly generated 8 puzzle configurations are used with statistics along with actual solution cost, where $x_1(n)$ is 5 for $h(n)$ which includes a linear combination

$$h(n) = C_1 x_1(n) + C_2 x_2(n)$$

$C \rightarrow$ cost
 $x \rightarrow$ parameter

where C_1 & C_2 are constants and $h(n)=0$ for goal state but it is not necessarily admissible (or) consistent.